Logging & Audit — Blanket Unredacted Capture, and a Stale Attribution Bug

InstantMudra Enterprises Admin Portal

⚠ AuditEntityListener always stamps "SYSTEM" — confirmed with exact code

```
Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
if (authentication != null && authentication.isAuthenticated()) return authentication.getName();
return "SYSTEM";
```

The real auth mechanism (AESFilter, Module 4) never populates Spring Security's SecurityContext — so this condition is never true, for any request, ever. createdBy/editedBy on every BaseEntity row are always "SYSTEM".

⚠ CRITICAL — blanket unredacted request/response capture across the entire PG & Payout vendor surface

GlobalApiLoggingAspect (@within(LoggableApi)) + VendorServiceLoggingAspect (@within(LoggableVendor))

What gets captured

```
request = objectMapper.writeValueAsString(args)
response = objectMapper.writeValueAsString(result)
```

Only non-serializable types are filtered out (HttpServletRequest/Response, MultipartFile) — **no field-level redaction of any kind. Bank account numbers, IFSC codes, vendor tokens, beneficiary PII — everything is captured in full, unmasked.**

Where it's applied — 13 files, confirmed by search

All 7 PG vendor impls (Razorpay, Unisus, Slpe, RunPaisa, NixaSoft, Cashfree, Shadval) + all 4 Payout vendor impls (Unisus, Shadval, RunPaisa, NixaSoft) + PaymentGateway-Controller + PayoutController.

This is the default, blanket logging mechanism for essentially the entire money-movement vendor surface — not a one-off, like the specific leaks in Module 11.

```
// VendorServiceLoggingAspect, in addition to the DB write:
log.info("Vendor Service Request : {}", request); log.info("Vendor Service Response : {}", response);
```

Same unredacted payload written to VendorLogs (DB) AND the application log file — the same double-write leak shape confirmed in Module 11.

VendorLogs.request / .response (TEXT columns) — everything, unmasked

⚠ VendorContext.clear() only clears half of what it tracks

```
private static final ThreadLocal<Boolean> serviceCalled = new ThreadLocal<>();
private static final ThreadLocal<String> vendorHolder = new ThreadLocal<>();
public static void clear() { serviceCalled.remove(); } // vendorHolder is NEVER removed
```

Both aspects call VendorContext.clear() in their finally block after every request — but clear() only removes serviceCalled. vendorHolder (the vendor ID) persists on the pooled servlet thread indefinitely.

On a reused thread, a later, unrelated request that never calls setVendorId() can inherit a PREVIOUS request's stale vendor ID — mis-attributing that log entry (im_vendor_id) to the wrong vendor.

By contrast: the de-duplication between the two aspects is sound

GlobalApiLoggingAspect only saves a log row if (!VendorContext.isServiceCalled()) — so when a @LoggableVendor service handled the request, the controller-level aspect steps aside rather than writing a duplicate row. Well-reasoned design, independent of the redaction and ThreadLocal issues above.

Sound design, by contrast Flagged bug Critical — confirmed systemic unredacted-data leak